

INTEGRIVE-100

User Manual

Standalone SDR Platform with Real-Time FPGA PHY

Version 1.0

Document ID: ITG-100-UM-001

Release Date: **September 2026**

Applicable to: Integrive-100 SDK v1.0 / FPGA Bitstream v1.0

Revision History

Version	Date	Author	Summary of Changes
1.0	September 2026	HTWAVE Co., Ltd.	Initial public release of the Integrive-100 User Manual, covering platform operation, SDK development, RF/PHY configuration, API usage, PHY research workflow, specifications, troubleshooting, and safety guidance.

Revision History	1
1. Introduction	4
1.1 Document Purpose	4
1.2 Scope and Audience	4
1.3 Document Conventions	5
2. Platform Overview	5
3. Software Architecture	6
3.1 Software Layer	6
3.2 FPGA Real-Time PHY	6
4. Role of the C/C++ Software	7
5. Getting Started	8
5.1 Requirements	8
5.2 SDK Package Contents	8
5.3 Installation	9
5.4 Application Development Workflow	9
5.4.1 Standard Application and Protocol Development	9
5.4.2 User-Developed PHY Functions	11
5.5 Building and Running Your First Application	11
5.5.1 Create the Example Source	12
5.5.2 Create the Makefile	13
5.5.3 Build the Application	13
5.5.4 Transfer the Application to Integrive-100	13
5.5.5 Run the Application	14
5.5.6 Development Cycle	14
6. Configuration Model	15
6.1 How to Read This Chapter	16
6.2 RF Configuration	16
6.2.1 Center Frequency	17
6.2.2 Sample Rate	17
6.2.3 RF Bandwidth	17
6.2.4 TX and RX Gain	18
6.2.5 TX/RX Channels	18
6.3 PHY Configuration	18
6.4 Modulation and Demodulation Configuration	18
6.5 OFDM Configuration	19
6.5.1 FFT Size	19
6.5.2 Cyclic-Prefix Length	19
6.5.3 Data and Pilot Configuration	19
6.6 FEC Configuration	20
6.7 Packet Configuration	20
6.8 PHY Control and User-Developed PHY Functions	21
6.9 PHY Monitoring	21
6.10 Derived OFDM Quantities	22
6.11 When a Parameter May Be Changed	23
6.12 Validation and Error Handling	23
6.13 Default and Reference Configurations	23
7. PHY Operation and Control	24
7.1 Basic Operation Flow	25

7.2 Opening and Closing the Device	25
7.3 Configuring the RF and PHY	26
7.4 Starting and Stopping the PHY	26
7.5 Transmitting and Receiving Data	27
7.6 Reconfiguration During Operation	27
7.7 User-Developed PHY Function Operation	28
7.8 Normal Application Shutdown	28
7.9 Recommended Application Sequence	29
8. Quick Start: First PHY Demo	29
8.1 Before You Begin	30
8.2 Obtain the Example Program	30
8.3 Review the PHY Configuration	31
8.4 Build the Example on the Host PC	32
8.5 Transfer the Example to Integrive-100	32
8.6 Run the Example	33
8.7 Verify PHY Operation	34
8.8 Stop the Demo	34
8.9 Next Steps	35
9. Integrive SDK API	35
9.1 Device Control	35
9.2 RF Configuration	36
9.3 PHY Configuration	36
9.3.1 Modulation	36
9.3.2 Forward Error Correction	37
9.4 PHY Operation	37
9.5 User-Developed PHY Functions	38
9.6 API Return Status	39
9.7 API Version and Compatibility	39
10. Error Handling	39
10.1 Checking API Results	40
10.2 Configuration Errors	40
10.3 Recommended Error-Handling Flow	41
10.4 Status-Code Reference	41
11. PHY Research Workflow	42
11.1 PHY Algorithm Research Workflow	42
11.2 Streaming PHY Processing	43
11.3 Development Scope	46
12. Specifications and Limitations	47
12.1 Operating Specifications	47
12.2 Current Limitations	47
13. Troubleshooting	48
13.1 Common Problems and Recommended Actions	48
13.2 Basic Troubleshooting Procedure	49
14. Regulatory and Safety Notice	49
14.1 RF Operation	50
14.2 Cabled TX/RX Testing	50
14.3 Handling Precautions	50
Glossary	51

1. Introduction

1.1 Document Purpose

This guide describes the software platform, programming interfaces, and development workflow provided with the Integrive-100 standalone real-time SDR platform.

Integrive-100 combines an FPGA-based real-time wireless PHY with a customized embedded Linux environment and C/C++ software interfaces. Time-critical PHY processing is executed in the FPGA, while the embedded Linux environment provides device control and configuration and supports C/C++ application and higher-layer protocol development.

The central design principle of Integrive-100 is:

Software configures the communication system; FPGA executes the real-time PHY.

The C/C++ software layer provides access to the configuration and control functions of the Integrive-100 platform, while the real-time TX and RX processing chains are executed in the FPGA.

Using the provided software interfaces and real-time PHY functions, researchers and developers can configure wireless communication parameters, conduct real-time RF experiments, and evaluate PHY algorithms without developing an entire SDR system from scratch. The customized embedded Linux environment also enables users to develop C/C++ applications and higher-layer communication protocols.

Multiple Integrive-100 units can be used to construct experimental wireless networks, enabling MAC and other communication protocols to be developed and evaluated in real RF environments.

This guide explains how to configure, program, operate, and evaluate the Integrive-100 software platform for wireless communication research and development.

1.2 Scope and Audience

This document is intended for researchers and developers who use Integrive-100 for real-time wireless communication research, development, and experimental validation.

The primary users include:

- **PHY researchers** who configure and evaluate modulation, FEC, OFDM, synchronization, framing, and other physical-layer functions.
- **Algorithm developers** who develop or evaluate PHY algorithms using the functions provided by the Integrive-100 platform and real-time RF signals.
- **MAC and protocol researchers** who use the customized embedded Linux environment and PHY interfaces to implement and evaluate communication protocols.
- **Network researchers** who use multiple Integrive-100 units to construct and evaluate experimental wireless networks.
- **Software developers** who develop C/C++ applications using the Integrive-100 software interfaces.

This guide focuses on the Integrive-100 software environment, software interfaces, PHY configuration and control, development and experimental workflows, and software-based interaction with the real-time FPGA PHY.

Detailed RF hardware specifications, hardware installation procedures, and internal FPGA implementation or modification procedures are outside the scope of this guide.

1.3 Document Conventions

Convention	Meaning
Monospaced text	API functions, type names, file names, configuration parameters, and shell commands
Stopped	The parameter can be changed only while the PHY is stopped
Runtime	The parameter can be changed while the PHY is running

Unless stated otherwise, frequencies are expressed in hertz (Hz), sample rates in samples per second (SPS), gains and signal-to-noise ratios in decibels (dB), and FFT and cyclic-prefix lengths in samples.

2. Platform Overview

Integrive-100 is a standalone real-time SDR platform that combines a customized embedded Linux environment, C/C++ software interfaces, an FPGA-based real-time PHY, and an RF transceiver.

As shown in Figure 2-1, time-critical PHY processing is performed in the FPGA, while the customized embedded Linux environment supports device configuration and control, C/C++ application development, and higher-layer communication protocol development.

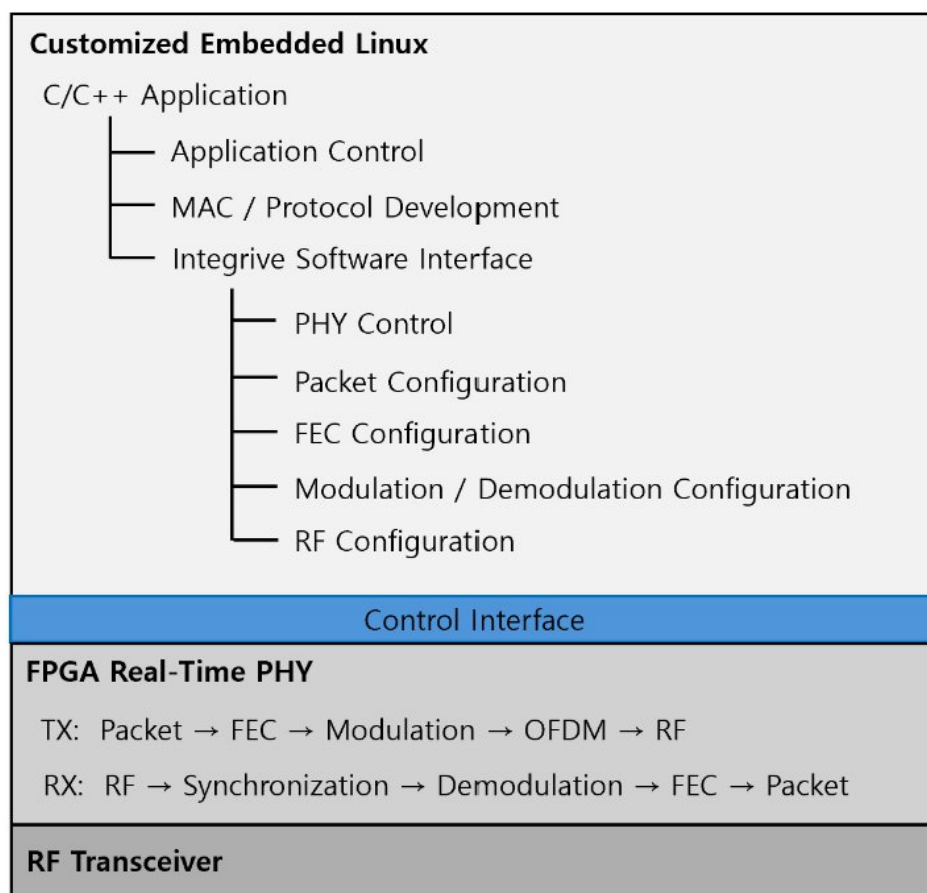


Figure 2-1. Integrive-100 Software and Hardware Architecture

The platform consists of three functional layers:

- **Customized Embedded Linux and Software Layer** — provides the runtime environment for user-developed C/C++ applications and communication protocols and access to the Integrive software interfaces for RF and PHY configuration and control.
- **FPGA Real-Time PHY** - Performs time-critical transmit and receive PHY processing.
- **RF Transceiver** - provides the RF transmit and receive interface.

Users configure and control the real-time PHY through the Integrive software interfaces without directly accessing the underlying FPGA hardware interfaces. Multiple Integrive-100 units can also be used to construct experimental wireless links and multi-node networks for real-time protocol development and evaluation.

3. Software Architecture

The Integrive-100 software architecture separates software-based configuration and application processing from real-time PHY processing. The customized embedded Linux environment provides the software execution environment, while the standard real-time PHY functions are executed in the FPGA.

3.1 Software Layer

The software layer runs on the embedded processor under the customized embedded Linux environment. It provides the environment for C/C++ application and higher-layer protocol development and provides software interfaces for configuring and controlling the FPGA PHY and RF transceiver.

The software layer supports:

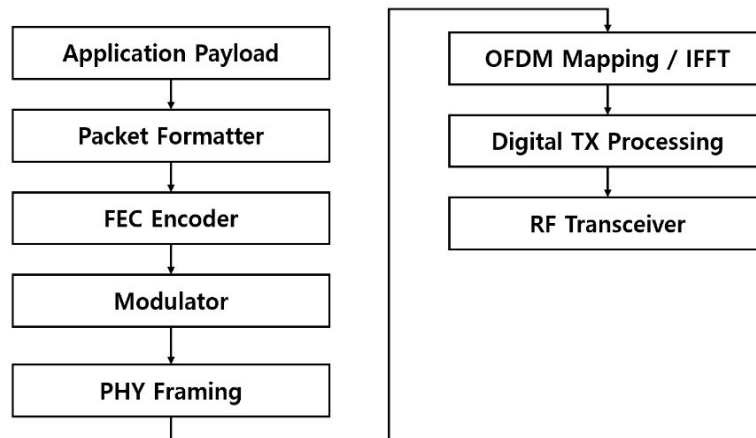
- Device initialization and shutdown
- RF frequency, sample rate, bandwidth, and gain configuration
- TX/RX channel configuration
- Modulation and demodulation configuration
- FEC configuration
- OFDM and framing configuration
- PHY start/stop and operating control
- C/C++ application development
- MAC and higher-layer protocol development

In the standard Integrive-100 PHY configuration, time-critical PHY processing is executed in the FPGA rather than in the embedded processor. User-developed PHY functions and their interaction with the real-time PHY processing path are described separately in Chapter 11.

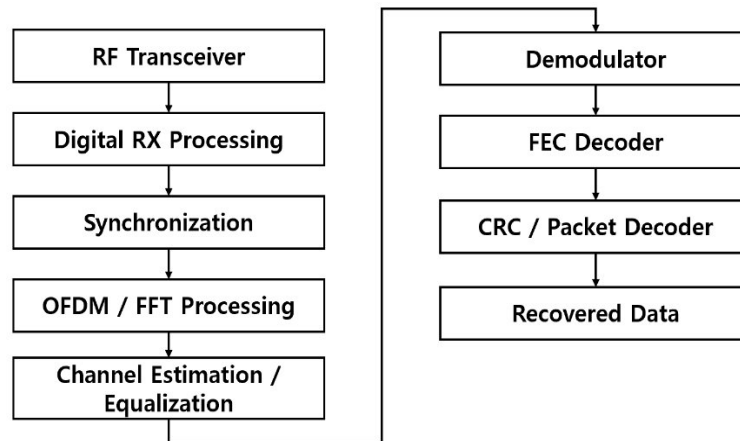
3.2 FPGA Real-Time PHY

The FPGA Real-Time PHY performs the time-critical transmit and receive processing required for real-time wireless communication, including packet processing, FEC, modulation and demodulation, OFDM processing, synchronization, channel estimation, and channel equalization.

The transmit data path is:



The receive data path is:



This architecture allows the embedded processor to handle software configuration, application logic, and higher-layer protocols, while the FPGA executes the standard real-time PHY processing functions.

4. Role of the C/C++ Software

The Integri-100 C/C++ API provides a software interface for configuring and controlling the RF transceiver and FPGA-based real-time PHY.

In the standard Integri-100 configuration, the processor configures and controls the communication system, while the FPGA executes the real-time PHY processing. PHY functions such as FEC, modulation/demodulation, OFDM processing, synchronization, and channel estimation/equalization are executed in the FPGA PL.

Through the Integri SDK, users can:

- Configure RF parameters such as frequency, sample rate, bandwidth, and gain.
- Configure supported PHY parameters such as modulation, FEC, and OFDM parameters.
- Start and stop PHY operation.
- Transmit and receive data through the real-time PHY.

These operations are performed through the C/C++ software interface without requiring direct

access to the underlying FPGA hardware interfaces.

This separation allows users to configure, control, and monitor the communication system through software while the FPGA performs the time-critical real-time PHY processing.

5. Getting Started

5.1 Requirements

Integrive-100 is shipped with a customized embedded Linux environment and the Integrive-100 kernel driver required for platform operation. Software development is performed on a host PC. User-developed applications and communication protocols are cross-compiled for the Integrive-100 ARM processor and built as ARM executables for execution on the Processing System (PS).

The following environment is required for software development.

Item	Requirement
Target platform	Integrive-100 board with an embedded ARM processor running the customized embedded Linux environment.
Integrive-100 kernel driver	The Integrive-100 kernel driver (integrive100.ko) is provided and preloaded on the platform before shipment.
Host development PC	Software development is performed on a host PC.
User application / protocol development	User-developed applications and communication protocols are cross-compiled on the host PC as ARM executables and transferred to Integrive-100 for execution on the PS.
Host build toolchain	GCC 9.
ARM cross-build toolchain	ARM cross-compiler compatible with the Integrive-100 target sysroot.

The standard Integrive-100 SDK does not require users to rebuild or modify the FPGA Programmable Logic (PL). Released FPGA configurations are provided by HTWAVE in binary form.

5.2 SDK Package Contents

The Integrive-100 SDK provides a public C API for user application and protocol development. The API is defined in `integrive.h`, while the Integrive runtime libraries, command-line utility, and FPGA configuration files are provided as pre-built binary files.

The SDK package includes the following files and directories:

Path	Contents
<code>include/integrive.h</code>	Public C API header defining the types, enumerations, and function interfaces available to user-developed applications and protocols.
<code>lib/libintegrive.so</code>	Pre-built shared runtime library implementing the Integrive software interface.
<code>lib/libintegrive.a</code>	Pre-built static library for linking user-developed applications and protocol modules.

bin/integrive-cli	Pre-built command-line utility for device access, configuration, and status monitoring.
bitstreams/	Released Integrive-100 FPGA configuration files, including boot.bin, provided in binary form.
examples/	Reference source-code examples demonstrating use of the Integrive C API.
docs/	API reference documentation and release notes.

The released FPGA configuration files are intended for use with the corresponding Integrive-100 software release. User modification or rebuilding of individual FPGA PHY blocks is not supported by the standard SDK.

5.3 Installation

Integrive-100 is shipped with the customized embedded Linux system and the required FPGA configuration pre-installed. Users normally need to install only the SDK on the host PC.

The Integrive-100 SDK is distributed through GitHub as a versioned package. In the commands below, `<version>` represents the SDK version number provided with each release.

Step 1. Download and Unpack the SDK

Download the Integrive-100 SDK package from GitHub and extract the archive:

```
tar -xzf integrive-sdk-<version>.tar.gz
cd integrive-sdk-<version>
```

After extraction, the `integrive-sdk-<version>` directory contains the SDK files and directories described in Section 5.2.

Step 2. Install the SDK

Run the installation script on the host PC:

```
sudo ./install.sh --prefix=/opt/integrive
```

The installation script installs the SDK under `/opt/integrive`.

Configure the environment so that the Integrive runtime library and command-line tools can be accessed:

```
export LD_LIBRARY_PATH=/opt/integrive/lib:$LD_LIBRARY_PATH
export PATH=/opt/integrive/bin:$PATH
```

For normal initial setup and operation, users do not need to modify or reload the FPGA configuration.

5.4 Application Development Workflow

5.4.1 Standard Application and Protocol Development

Integrive-100 uses a host-based development model. User applications and communication protocols are developed and built on a host PC and then transferred to the Integrive-100 platform

for execution.

Integrive-100 is not intended for local software development using a directly connected monitor and keyboard. The customized embedded Linux environment provides the runtime environment for the SDR platform, while software development is performed on the host PC.

User-developed applications and communication protocols are cross-compiled for the Integrive-100 ARM processor and built as **ARM executables**. The resulting executable files are transferred to Integrive-100 and run on the Processing System (PS).

Linux kernel modules (.ko) are used separately when Linux driver-level functions need to be developed or modified. Driver development is not part of the normal application and protocol development workflow.

The basic development workflow is shown in Figure 5-1.

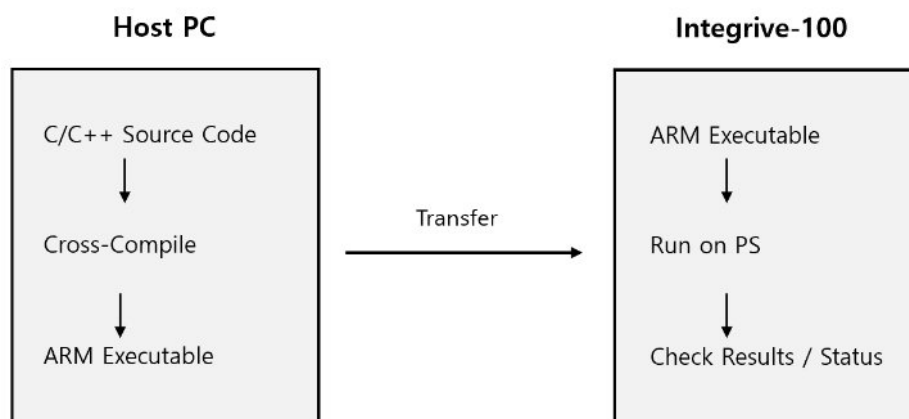


Figure 5-1. Integrive-100 Application Development Workflow

In a typical development cycle:

1. Write or modify the C/C++ source code on the host PC.
2. Cross-compile the source code for the Integrive-100 ARM target.
3. Build the application or protocol as an ARM executable.
4. Transfer the executable to Integrive-100.
5. Run the executable on the Processing System (PS).
6. Check the execution results and application status.
7. Modify the source code and repeat the development cycle as required.

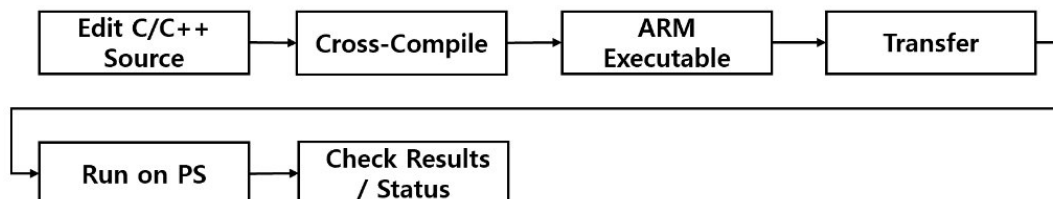


Figure 5-2. Host-Based Application and Protocol Development Workflow

This workflow is used for both user applications and user-developed communication protocols.

5.4.2 User-Developed PHY Functions

In addition to applications and communication protocols, selected PHY functions can be developed in C and executed in the Integrive-100 customized embedded Linux environment.

The standard real-time PHY functions provided with Integrive-100, including FEC, modulation / demodulation, OFDM processing, synchronization, and channel estimation / equalization, are executed in the FPGA Programmable Logic (PL).

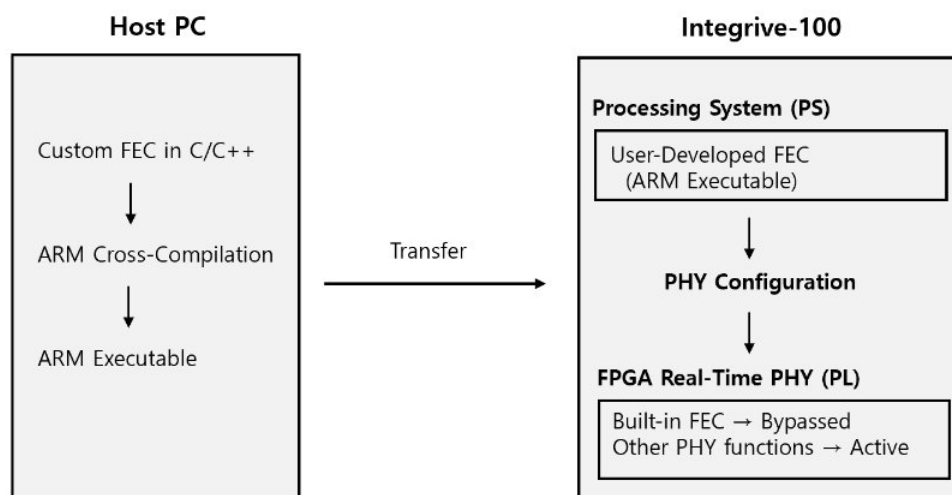


Figure 5-3. User-Developed PHY Function Execution Model

Figure 5-3 illustrates the execution model using a custom FEC algorithm as an example. The user develops the FEC algorithm in C/C++ on the host PC and cross-compiles it as an ARM executable for execution on the Integrive-100 Processing System (PS). The executable is then transferred to Integrive-100 and run on the PS.

When the user-developed FEC function is selected, the corresponding built-in FEC function in the FPGA PL is bypassed, while the remaining FPGA PHY functions continue to operate normally. The user-developed FEC function processes the data transferred from the FPGA PHY and returns the processed data to the FPGA processing chain.

This approach allows a user-developed PHY function to operate together with the remaining Integrive-100 PHY functions and to be evaluated using real RF signals without rebuilding the complete FPGA PHY. Linux kernel modules ([.ko](#)) are used when developing or modifying Linux driver-level functions and are separate from the normal PS-based user PHY development workflow. The processing model for user-developed PHY functions is described in Chapter 11.

Note: The standard Integrive-100 SDK does not support rebuilding or replacing individual FPGA PHY blocks in the PL.

5.5 Building and Running Your First Application

This section demonstrates how to build and run a simple C/C++ application on Integrive-100. The example follows the host-based development workflow described in Section 5.4 and is cross-compiled as an ARM executable for execution on the Integrive-100 Processing System (PS).

The purpose of this example is to verify the basic host-based development environment before proceeding to PHY application development.

The example follows this sequence:

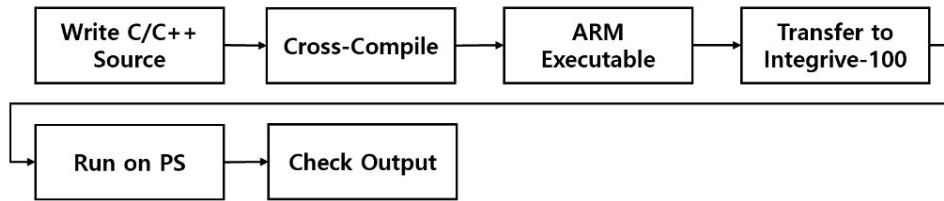


Figure 5-4. Building and Running a Simple Integrive-100 Application

5.5.1 Create the Example Source

On the host PC, create a working directory and a source file named `first_demo.c`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int count = 3;
    int i;

    if (argc > 1) {
        count = atoi(argv[1]);
    }

    printf("FIRST_DEMO: started, count=%d\n", count);

    for (i = 0; i < count; i++) {
        printf("FIRST_DEMO: hello %d/%d\n", i + 1, count);
        usleep(100000);
    }

    printf("FIRST_DEMO: completed\n");

    return 0;
}
```

The program prints a sequence of messages and then terminates. The optional command-line argument specifies how many messages are generated. If no argument is provided, the default value is 3.

For example:

```
./first_demo 10
```

prints ten messages before the application terminates.

This simple program is used to verify that the host cross-compilation environment, file-transfer

path, and Integrive-100 PS runtime environment are operating correctly.

5.5.2 Create the Makefile

In the same directory as `first_demo.c`, create a file named `Makefile`:

```
CROSS ?= arm-linux-gnueabi-
CC := $(CROSS)gcc
CFLAGS ?= -O2 -Wall

TARGET = first_demo
SRC = first_demo.c

all:
    $(CC) $(CFLAGS) -o $(TARGET) $(SRC)

clean:
    rm -f $(TARGET)
```

This Makefile cross-compiles `first_demo.c` for the Integrive-100 ARM processor and generates the executable file:

```
first_demo
```

The compiler prefix may be adjusted according to the ARM cross-compilation toolchain installed on the host PC.

5.5.3 Build the Application

Configure the ARM cross-compiler on the host PC if the toolchain is not already included in the system `PATH`:

```
export PATH=/tools/Xilinx/Vitis/2022.2/gnu/aarch32/lin/gcc-arm-linux-gnueabi/bin:$PATH
```

Then build the application:

```
make CROSS=arm-linux-gnueabi-
```

The Makefile cross-compiles `first_demo.c` for the Integrive-100 ARM processor and generates the following executable:

```
first_demo
```

Verify that the executable has been generated:

```
ls -l first_demo
file first_demo
```

The `file` command can be used to confirm that the generated program is an ARM executable suitable for the Integrive-100 target environment.

5.5.4 Transfer the Application to Integrive-100

If Integrive-100 is connected to the host PC through a network, transfer the executable using `scp`:

```
scp first_demo root@<board-ip>:/root/
```

Replace <board-ip> with the IP address assigned to the Integrive-100 unit.

For example:

```
scp first_demo root@192.168.1.10:/root/
```

The transferred executable will be available as:

```
/root/first_demo
```

Network transfer using `scp` provides a simple workflow when network connectivity is available.

5.5.5 Run the Application

Connect to Integrive-100 through SSH:

```
ssh root@<board-ip>
```

Run the application:

```
/root/first_demo 10
```

The command-line argument 10 specifies the number of messages to be generated by the application.

A successful execution produces output similar to the following:

```
FIRST_DEMO: started, count=10
FIRST_DEMO: hello 1/10
FIRST_DEMO: hello 2/10
...
FIRST_DEMO: hello 10/10
FIRST_DEMO: completed
```

If no command-line argument is specified, the application uses the default value of 3:

```
/root/first_demo
```

The application runs on the Integrive-100 Processing System (PS) and terminates after the specified number of messages has been printed.

5.5.6 Development Cycle

After the first example has been successfully executed, the same host-based development workflow can be used for user-developed applications and communication protocols.

Users can modify the source code on the host PC, cross-compile it for the Integrive-100 ARM processor, transfer the resulting executable to Integrive-100, and run it on the Processing System (PS). The results can then be evaluated and the development cycle repeated as required.

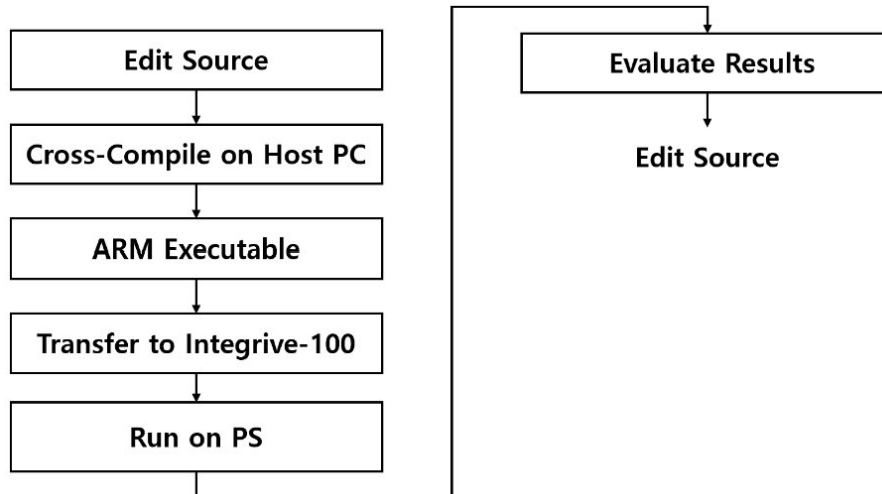


Figure 5-5. host-based development cycle

This host-based development cycle allows users to develop and evaluate their own software functions on the Integrive-100 platform while using the FPGA-based real-time PHY provided by the system.:

5.6 Verifying the Development Environment

After completing the first application example, verify that:

- [first_demo](#) is successfully built on the host PC.
- The executable is successfully transferred to Integrive-100.
- The application runs on the Integrive-100 Processing System (PS).
- The expected [FIRST_DEMO](#) messages are displayed.
- The application terminates normally.

Successful completion of these steps confirms that the host build environment, ARM cross-compilation toolchain, file-transfer path, and Integrive-100 PS runtime environment are operating correctly.

6. Configuration Model

Integrive-100 provides a software configuration interface for controlling the RF transceiver and the FPGA-based real-time PHY. User applications configure the platform through the Integrive Software Interface rather than directly accessing FPGA registers or RF-device control registers.

The configuration interface covers the following major functions:

- RF configuration
- Modulation and demodulation configuration
- OFDM configuration
- FEC configuration
- Packet configuration

- PHY control and processing-path configuration

Configuration parameters may be changed either while the PHY is stopped or, for selected RF parameters, while the PHY is running.

This chapter describes the supported configuration parameters, configuration rules, and their relationship to the Integrive SDK.

6.1 How to Read This Chapter

Configuration parameters are described using the following terms.

Term	Meaning
Default	A parameter value automatically applied by the released Integrive-100 hardware/SDK configuration.
Reference	An example configuration used in this guide for explanation or experiments. A reference value is not necessarily a system default.
Stopped	The parameter may be changed only while the PHY is stopped.
Runtime	The parameter may be changed while the PHY is running.
Derived	The value is calculated from other configuration parameters and is not configured directly.

Unless otherwise stated, frequencies are expressed in hertz (Hz), sample rates in samples per second (SPS), gains in decibels (dB), and FFT and cyclic-prefix lengths in samples.

Invalid or unsupported configuration requests are rejected rather than silently adjusted. The previously valid configuration remains unchanged.

The API declarations in `integrive.h` included with the released SDK are the authoritative definitions of the software interface.

6.2 RF Configuration

The RF configuration interface controls the RF transceiver parameters used by the transmit and receive paths.

The current SDK provides RF configuration functions including:

```
integrive_rf_set_frequency()
integrive_rf_set_sample_rate()
integrive_rf_set_bandwidth()
integrive_rf_set_gain()
integrive_rf_set_gain_mode()
integrive_rf_enable_channel()
```

The `dir` argument selects the TX or RX direction, while the `channel` argument selects the RF channel where applicable.

6.2.1 Center Frequency

The center, or local-oscillator (LO), frequency determines the RF operating frequency.

Item	Value
API	integrive_rf_set_frequency()
Unit	Hz
Supported range	300 MHz to 6 GHz
Default	915 MHz
Change in	Stopped

6.2.2 Sample Rate

The sample rate determines the digital sampling rate used by the PHY processing chain.

Item	Value
API	integrive_rf_set_sample_rate()
Unit	SPS
Maximum	61.44 MSPS
Default	60 MSPS
Change in	Stopped

The sample rate affects derived OFDM quantities such as subcarrier spacing and OFDM symbol duration.

Both RF channels in the same direction share the sample-rate clock. Therefore, the sample-rate setting applies to the selected TX or RX direction rather than to an individual channel.

6.2.3 RF Bandwidth

The RF bandwidth controls the analog RF filtering applied to the selected channel.

Item	Value
API	integrive_rf_set_bandwidth()
Unit	Hz
Maximum	20 MHz
Default	20 MHz
Change in	Stopped

The relationship between sample rate and usable OFDM bandwidth is determined by the selected waveform and OFDM configuration rather than by a separate Integrive-100 product-specific constraint.

6.2.4 TX and RX Gain

TX and RX gain are configured through:

```
integrive_rf_set_gain(dev, dir, channel, gain_db);
```

The dir argument selects whether the setting applies to the TX or RX path.

RX gain mode can be selected through:

```
integrive_rf_set_gain_mode();
```

The supported gain range extends up to approximately 70 dB. RX operation supports automatic gain control (AGC), which is used as the default RX gain mode.

Because TX and RX gain settings are handled differently by the RF transceiver, this guide does not specify a common lower gain limit or TX default value.

When the TX and RX ports are directly connected for a cabled loopback experiment, appropriate attenuation must be used to prevent receiver overload.

6.2.5 TX/RX Channels

Integrive-100 supports up to:

- 2 TX channels
- 2 RX channels

The default configuration uses:

- 1 TX channel
- 1 RX channel

Individual TX and RX channels can be enabled or disabled using:

```
integrive_rf_enable_channel();
```

Channel indices are provided through the corresponding RF API calls.

6.3 PHY Configuration

The FPGA PHY is configured using the `integrive_phy_config_t` structure and applied through:

```
integrive_phy_configure(dev, &cfg);
```

The PHY configuration includes parameters for modulation, coding, OFDM processing, and other PHY functions.

Parameters that change the structure of the PHY processing chain are configured while the PHY is **Stopped**.

6.4 Modulation and Demodulation Configuration

Integrive-100 provides FPGA-based real-time modulation and demodulation.

The current PHY supports:

Modulation	Bits per Symbol
------------	-----------------

BPSK	1
QPSK	2
16-QAM	4
64-QAM	6
256-QAM	8

The modulation mode is selected through the corresponding field of `integrive_phy_config_t`.

A fixed Gray-coded constellation mapping is used for the supported modulation schemes.

The receive PHY performs demodulation according to the configured modulation mode.

Modulation and demodulation configuration is changed while the PHY is **Stopped**.

6.5 OFDM Configuration

OFDM configuration determines the frequency-domain structure and timing of the PHY waveform.

6.5.1 FFT Size

The FFT size is configured through the corresponding PHY configuration parameter.

Item	Value
Unit	samples
Default	1024
Change in	Stopped

6.5.2 Cyclic-Prefix Length

The `cyclic-prefix` length is configured through the corresponding PHY configuration parameter.

Item	Value
Unit	samples
Default	26
Change in	Stopped

The `cyclic-prefix` length may be selected according to the expected channel delay spread and the requirements of the experiment.

6.5.3 Data and Pilot Configuration

The data and pilot allocation can be defined by the user according to the requirements of the PHY experiment.

Based on the configured data and pilot allocation, the PHY automatically determines the corresponding number of symbols and active subcarriers required for the OFDM configuration.

A reference configuration used in this guide is:

Subcarrier Type	Reference Count
-----------------	-----------------

Data subcarriers	264
Pilot subcarriers	24
Total used subcarriers	288

These values represent a **reference configuration** and are not fixed hardware limits.

The DC subcarrier is nulled in the current implementation. Pilot and data subcarriers use the same signal amplitude.

The detailed pilot sequence and internal allocation mechanism are not specified in this release.

6.6 FEC Configuration

Integrive-100 provides Reed–Solomon (RS) forward error correction in the FPGA PHY.

The current SDK provides the following coding modes:

```
INTEGRIVE_CODE_NONE
INTEGRIVE_CODE_RS_255_215
INTEGRIVE_CODE_RS_765_645
```

The coding mode is selected as part of the PHY configuration.

The Reed–Solomon FEC block supports 6-, 8-, and 10-bit RS symbol widths. The appropriate symbol width is automatically selected by the FEC block according to the configured input length.

For the RS(255,215) configuration:

- each codeword contains 255 RS symbols,
- 215 symbols carry payload data, and
- up to 20 erroneous RS symbols can be corrected per codeword.

The RS(765,645) configuration is implemented using three sequential RS(255,215) codewords. No interleaving is used.

FEC configuration is changed while the PHY is **Stopped**.

6.7 Packet Configuration

Integrive-100 supports packet-based transmission and reception through the real-time FPGA PHY.

The current packet characteristics are:

Parameter	Current Configuration
Payload length	100–6450 bytes
Default payload length	1500 bytes
CRC	CRC-32
CRC coverage	Header + payload
Preamble	Fixed

Preamble length	Not specified in this release
Test pattern	Not specified in this release

The supported payload range is enforced by the FPGA PHY implementation.

Payload transmission and decoded-packet reception are performed through the corresponding PHY send and receive interfaces.

The detailed preamble structure and internal test patterns are not specified in this release.

6.8 PHY Control and User-Developed PHY Functions

Under standard operation, the complete real-time PHY processing chain is executed using the FPGA PHY provided with Integrive-100.

For selected PHY experiments, Integrive-100 also allows one FPGA PHY processing function to be replaced by a user-developed C function running on the Processing System (PS).

The current SDK provides interfaces for selecting the implementation of a PHY processing stage and associating a user-developed function with that stage.

The replaceable PHY processing stages include functions such as:

- Encoding
- Modulation
- Synchronization
- Equalization
- Demodulation
- Decoding

Only **one FPGA PHY processing block at a time** can be replaced by a user-developed PS function in the current implementation.

The user-developed function operates together with the remaining Integrive-100 PHY processing chain. This allows researchers to evaluate their own PHY algorithms using real RF signals without rebuilding the complete FPGA PHY.

The detailed streaming PHY processing model is described in Chapter 11.

The detailed internal data representation and FPGA/PS boundary implementation are not specified in this release.

6.9 PHY Monitoring

Integrive-100 provides PHY measurement capabilities for evaluating communication performance, including:

- SNR
- EVM
- BER
- PER
- Packet counters

The current integrive.h includes the `integrive_phy_get_stats()` interface for accessing PHY statistics. However, this API is **not implemented in the current SDK release**.

Software access to these PHY statistics through `integrive_phy_get_stats()` is therefore reserved for a future SDK update and will be documented when the function becomes available.

6.10 Derived OFDM Quantities

Several useful OFDM quantities can be calculated directly from the configured parameters.

The subcarrier spacing is:

$$\Delta f = \frac{f_s}{N_{\text{FFT}}}$$

where f_s is the sample rate and N_{FFT} is the FFT size.

The OFDM symbol duration including the cyclic prefix is:

$$T_{\text{sym}} = \frac{N_{\text{FFT}} + N_{\text{CP}}}{f_s}$$

The corresponding OFDM symbol rate is:

$$R_{\text{sym}} = \frac{1}{T_{\text{sym}}}$$

For the default configuration:

- Sample rate = 60 MSPS
- FFT size = 1024
- CP length = 26 samples

the resulting values are:

Quantity	Value
Subcarrier spacing	58.59375 kHz
OFDM symbol duration	17.50 μ s
OFDM symbol rate	approximately 57,142.9 symbols/s
CP overhead	approximately 2.48%

The theoretical coded PHY payload rate may be estimated as:

$$R_b = N_{\text{data}} \times b \times R_c \times R_{\text{sym}}$$

where:

- N_{data} = number of data subcarriers,
- b = bits per modulation symbol,
- R_c = FEC code rate.

The resulting PHY data rate therefore depends on the selected data-subcarrier allocation, modulation mode, and FEC configuration.

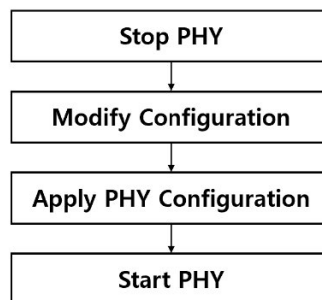
6.11 When a Parameter May Be Changed

Configuration parameters are classified according to when they may be changed.

Change State	Description
Stopped	The PHY must be stopped before the parameter is changed.
Runtime	The parameter may be changed while the PHY is running.
Derived	The value is calculated from other parameters and is not configured directly.

Parameters that alter the digital PHY processing configuration, including sample rate, modulation, FEC, FFT size, cyclic-prefix length, and processing-stage selection, are changed while the PHY is **Stopped**.

The typical reconfiguration sequence is:



6.12 Validation and Error Handling

Configuration requests are validated by the Integrive software interface. Unsupported or invalid requests are rejected rather than silently modified.

The current SDK uses `integrive_status_t` for API return status. `INTEGRIVE_OK` indicates successful completion, while negative values indicate errors.

A human-readable description of an error can be obtained using:

```
integrive_strerror(status);
```

Detailed status codes and error handling are described in Chapter 10.

6.13 Default and Reference Configurations

A **default configuration** is a configuration automatically provided by the released Integrive-100 platform.

The current SDK provides:

```
integrive_phy_get_default_config()
```

for obtaining the baseline PHY configuration.

The principal default values used in this guide are:

Parameter	Default
Center frequency	915 MHz
Sample rate	60 MSPS
RF bandwidth	20 MHz
RX gain mode	AGC
Active channels	1 TX + 1 RX
FFT size	1024
CP length	26 samples
Packet payload	1500 bytes

A **reference configuration**, by contrast, is an example configuration used to explain operation or conduct a particular experiment. Reference values do not redefine the platform defaults.

For example, the OFDM allocation of:

- 264 data subcarriers
- 24 pilot subcarriers

is a **reference configuration**, not a fixed hardware configuration.

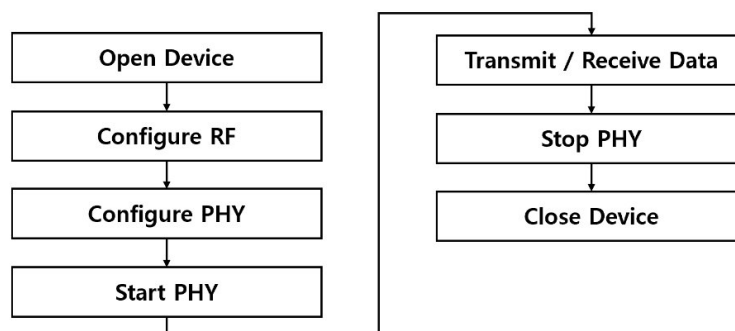
This distinction is maintained throughout this guide.

7. PHY Operation and Control

Integrive-100 provides a straightforward software-controlled operating model for configuring and running the real-time FPGA PHY.

A typical application opens the device, configures the RF and PHY parameters, starts the PHY, exchanges data, and then stops and closes the device.

The basic operating sequence is:



This chapter describes the operating sequence and the corresponding Integrive SDK interfaces. Detailed configuration parameters are described in Chapter 6.

7.1 Basic Operation Flow

A typical Integrive-100 application follows six basic steps:

1. Open the Integrive-100 device.
2. Configure the required RF parameters.
3. Configure the FPGA PHY.
4. Start PHY operation.
5. Transmit and/or receive data.
6. Stop the PHY and close the device.

The RF and PHY configurations should be completed before starting the PHY when the selected parameters require the PHY to be in the **Stopped** condition.

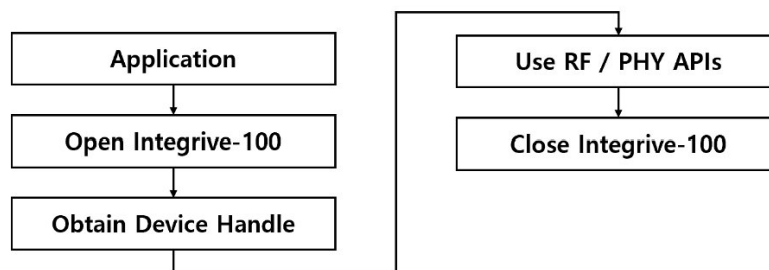
Selected RF parameters that support **Runtime** changes may be adjusted while the PHY is operating, as described in Chapter 6.

7.2 Opening and Closing the Device

Before RF or PHY functions can be accessed, the application opens the Integrive-100 device and obtains a device handle.

The device handle is then passed to the Integrive SDK functions used throughout the application.

The general operating model is:



When the application has completed its operation, the device should be closed using the corresponding SDK interface.

The exact function declarations and argument definitions are provided in [integrive.h](#) and are described in the API Reference in Chapter 9.

7.3 Configuring the RF and PHY

After opening the device, the application configures the RF and PHY according to the requirements of the experiment.

Typical RF configuration includes:

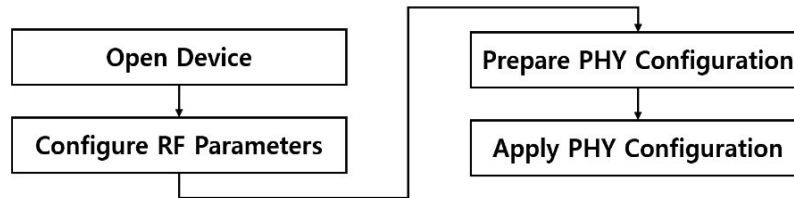
- Center frequency
- Sample rate
- RF bandwidth
- TX/RX gain
- TX/RX channel selection

RF parameters are configured using the [integrive_rf_*](#) interfaces described in Chapter 6.

The FPGA PHY is configured using an `integrive_phy_config_t` structure and:

```
integrive_phy_configure(dev, &cfg);
```

A typical configuration sequence is:



The SDK also provides:

```
integrive_phy_get_default_config()
```

to obtain the default PHY configuration as a starting point.

The application may then modify the required fields before applying the configuration.

7.4 Starting and Stopping the PHY

After the RF and PHY configurations have been completed, real-time PHY operation is started using:

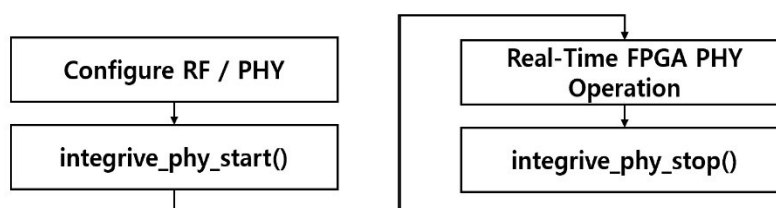
```
integrive_phy_start();
```

Once started, the FPGA performs the configured real-time transmit and receive PHY processing.

PHY operation is stopped using:

```
integrive_phy_stop();
```

The basic control sequence is:



Parameters classified as **Stopped** in Chapter 6 should be modified only after the PHY has been stopped.

7.5 Transmitting and Receiving Data

During PHY operation, application data can be transferred through the packet-oriented PHY interface.

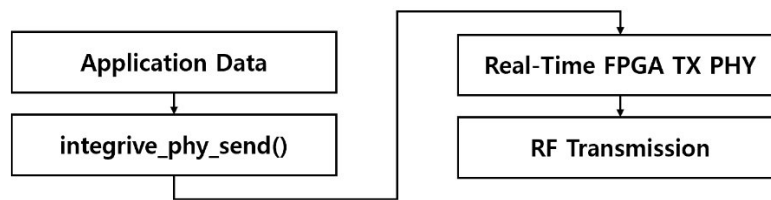
Transmission is performed using:

```
integrive_phy_send();
```

The application provides the data to be transmitted, and the FPGA PHY performs the configured

real-time processing required for RF transmission.

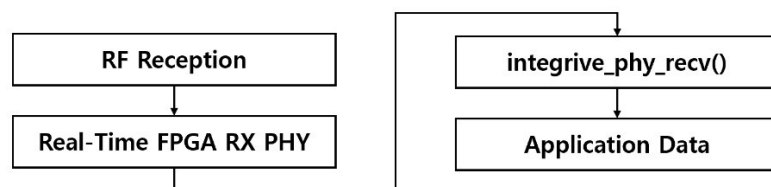
The transmit path is conceptually:



Received data is obtained using:

```
integrive_phy_rcv();
```

The FPGA PHY performs the real-time receive processing before delivering recovered packet data to the application.



The internal PHY processing stages for transmission and reception are described in Chapter 3.

7.6 Reconfiguration During Operation

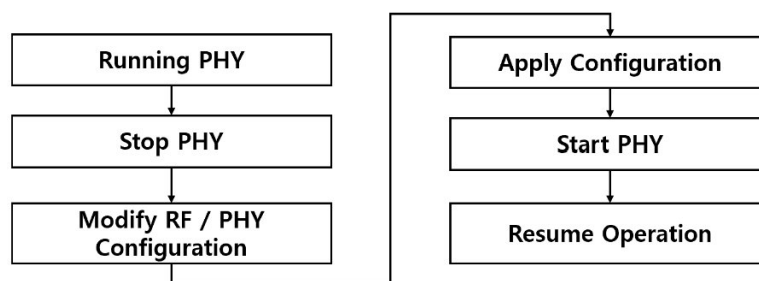
Integrive-100 distinguishes between parameters that may be changed during PHY operation and parameters that require the PHY to be stopped.

As described in Chapter 6:

- **Runtime** parameters may be changed while the PHY is running.
- **Stopped** parameters require the PHY to be stopped before modification.

For example, parameters that modify the digital PHY structure, such as modulation, FEC, FFT size, cyclic-prefix length, or processing-stage selection, require the PHY to be stopped.

The normal reconfiguration procedure is:



This approach ensures that structural PHY parameters are not changed while the FPGA is actively processing the current waveform.

7.7 User-Developed PHY Function Operation

When a user-developed PHY function is used, the corresponding processing stage is selected before the PHY is started.

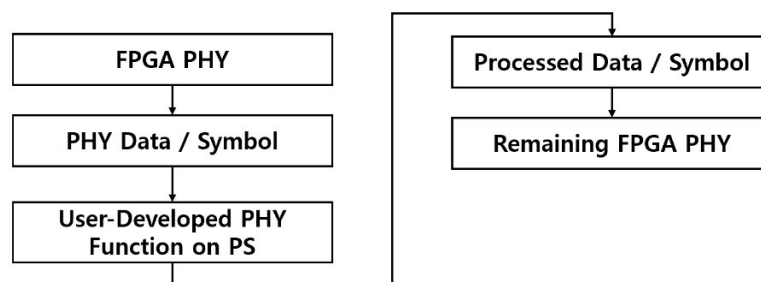
The current SDK provides interfaces for:

- Selecting the implementation of a PHY processing stage
- Associating a user-developed function with the selected stage
- Returning the stage to the standard FPGA implementation

In the current implementation, **one FPGA PHY processing block at a time** can be replaced by a user-developed function running on the Processing System (PS).

The remaining PHY processing functions continue to execute in the FPGA.

Conceptually:



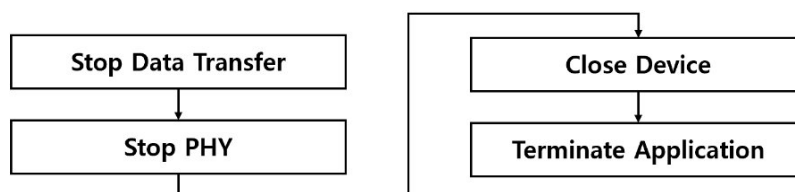
The processing-stage configuration is completed while the PHY is **Stopped**.

The detailed streaming processing model and development workflow for user-developed PHY functions are described in Chapter 11.

7.8 Normal Application Shutdown

Before terminating an application, the PHY should be stopped and the Integrive-100 device should be closed.

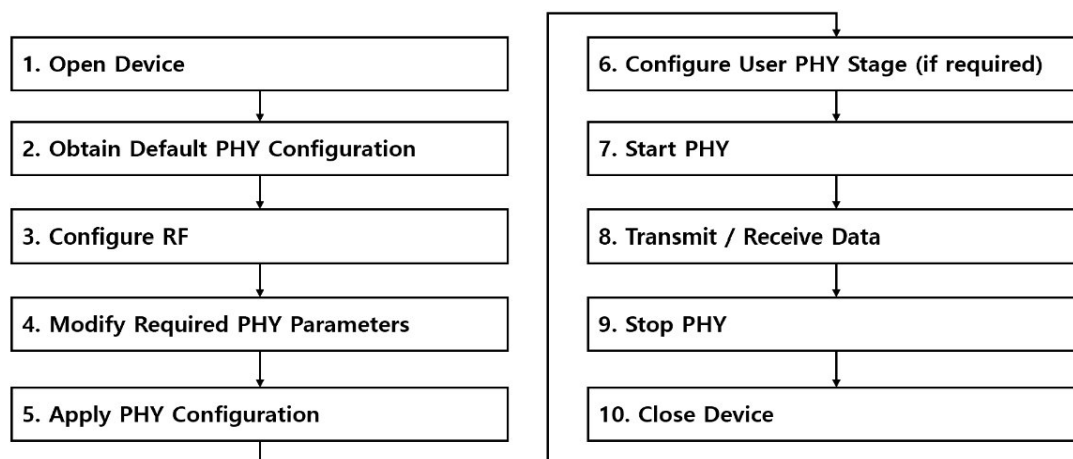
The normal shutdown sequence is:



This provides a controlled termination of the current PHY operation and releases the software resources associated with the device.

7.9 Recommended Application Sequence

The complete application sequence can therefore be summarized as follows:



This sequence provides the basic software operating model for Integrive-100 applications.

The Quick Start procedure for obtaining, building, and running a released SDK example is provided in Chapter 8.

8. Quick Start: First PHY Demo

This chapter provides a quick-start procedure for building and running an Integrive-100 PHY application. It first introduces the general development and execution workflow, and then demonstrates the procedure using a user-developed FEC decoder running on the ARM processor while the remaining PHY functions continue to operate in the FPGA.

Complete example programs are provided and maintained with the Integrive SDK, while this User Manual describes the procedures required to obtain, build, transfer, run, and verify an example using the released Integrive SDK.

The basic Quick Start workflow is:

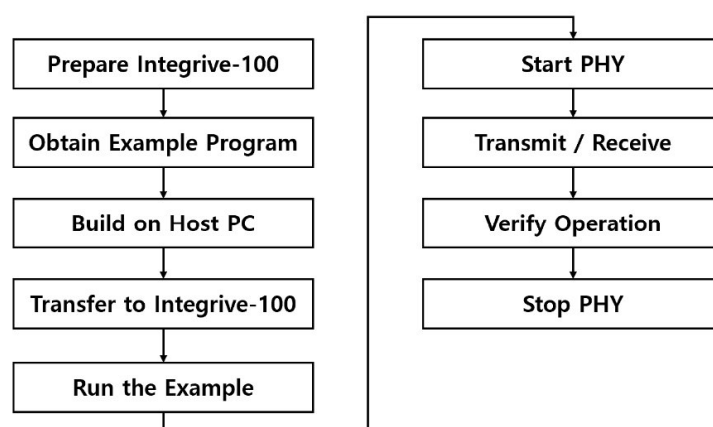


Figure 8-1. Basic Quick Start workflow for an Integrive-100 PHY application.

8.1 Before You Begin

Before running the first PHY demo, verify that:

Integrive-100 is powered and running the preinstalled customized embedded Linux.

- The host PC is connected to Integrive-100 through the development network.

- The Integrive SDK is installed on the host PC.
- The ARM cross-compilation environment described in Chapter 5 is available.
- [integrive100.ko](#) is loaded on Integrive-100.
- The required FPGA bitstream is loaded.
- The required RF connection or test setup has been prepared.

For conducted RF testing, appropriate attenuation should be used between the TX and RX paths to prevent receiver overload.

The host PC is used for software development and cross-compilation. Real-time PHY processing is performed on Integrive-100 after the application has been transferred to the platform.

8.2 Obtain the Example Program

Example programs for Integrive-100 are maintained in the Integrive-100 GitHub repository. The examples are provided separately from this User Manual so that they can be updated together with the SDK release.

Users should obtain an example corresponding to the installed SDK and FPGA bitstream release.

For the Quick Start demonstration in this chapter, the **FEC-on-PS example** is used. The example replaces the FPGA FEC decoder with a Reed–Solomon decoder implemented in C and executed on the ARM processor, while the remaining receive PHY functions continue to operate in the FPGA.

The FEC-on-PS example used for the first PHY demo is provided in the following directory of the Integrive-100 GitHub repository:

[examples/fec_on_ps/](#)

The primary source files are:

File	Description
fec_on_ps.c	Main example application and PS-side FEC callback
ps_rs_decode.c	Reed–Solomon decoder executed on the ARM processor

The main application configures the RX substitution span, registers the PS-side decoder callback, starts the PHY, and verifies the decoding results.

The example source code also provides a practical starting point for developing other Integrive-100 applications and user-developed PHY functions.

8.3 Review the PHY Configuration

Before building or running an example, review its RF and PHY configuration.

Typical configuration parameters include:

- Center frequency
- Sample rate

- RF bandwidth
- TX/RX gain
- TX/RX channel selection
- Modulation
- FEC
- FFT size
- Cyclic-prefix length
- Packet payload length
- Data and pilot allocation

For the first experiment, it is recommended to use the configuration supplied with the example rather than changing multiple parameters simultaneously.

The FEC-on-PS example is used with a PHY configuration based on:

- Modulation: 256-QAM
- FFT size: 1024
- Cyclic prefix: 26 samples
- Data subcarriers: 765
- Pilot subcarriers: 11
- FEC: RS(255,215)

The example SDK configuration uses a center frequency of 2.484 GHz, a sample rate of 15.36 MSPS, and an RF bandwidth of 12.4 MHz.

These values are specific to the example configuration and loaded FPGA bitstream. They do not represent fixed RF settings of the Integrive-100 platform. RF parameters that are configurable by the user should be set within the supported operating conditions described in Chapter 6.

8.4 Build the Example on the Host PC

Integrive-100 applications are developed and built on the host PC.

The development model is:

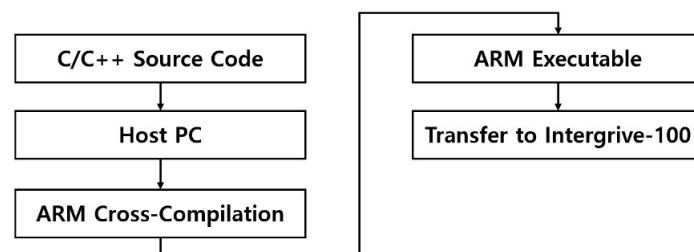


Figure 8-2. Host-based development and cross-compilation workflow.

The host development environment and cross-compilation procedure are described in Chapter 5.

For the FEC-on-PS example, the source code specifies the following build procedure:

```
make CROSS=arm-linux-gnueabi-
```

The resulting ARM executable is transferred to Integrive-100 and executed on the platform.

Users should follow the build instructions supplied with the corresponding example because build options may differ between SDK releases or examples.

Any build errors should be resolved on the host PC before transferring the executable to Integrive-100.

8.5 Transfer the Example to Integrive-100

After a successful build, transfer the resulting executable to Integrive-100 over the development network.

As described in Chapter 5, files can be transferred to the board using [scp](#).

For example, the general transfer procedure is:

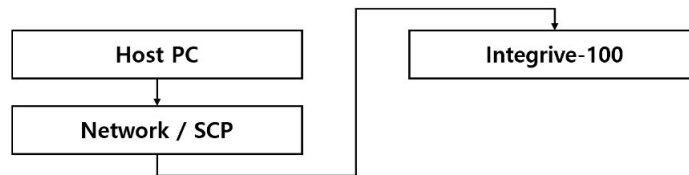


Figure 8-3. Transfer of a cross-compiled application to Integrive-100.

The exact filename depends on the selected example.

After transfer, connect to Integrive-100 through SSH and verify that the executable and any required test files are present.

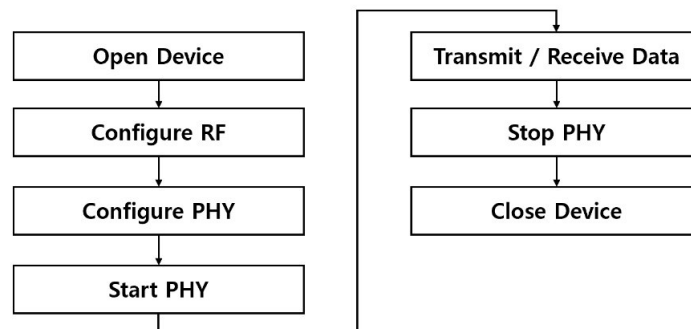
The exact executable name and destination directory depend on the selected example.

For the FEC-on-PS example, a golden payload file may also be provided for byte-exact verification of the decoded data. The example searches for the reference payload using a command-line argument, the [GOLDEN](#) environment variable, or the default [/root/golden6450.bin](#) file.

8.6 Run the Example

Run the example directly on Integrive-100 according to the instructions supplied with the example program.

The general application sequence is:



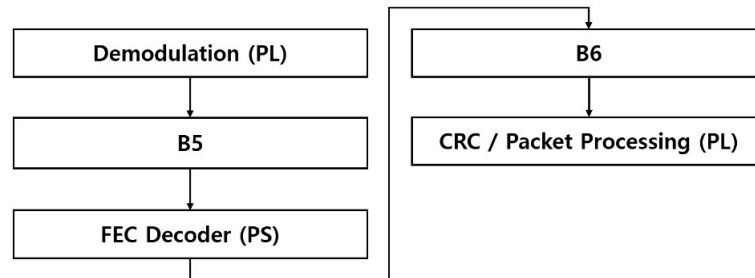
This sequence follows the Integrive-100 operating model described in Chapter 7.

For the FEC-on-PS example, the general command format is:

```
/tmp/fec_on_ps [packets]
```

If the packet count is not specified, the program uses its predefined default value.

The example configures the receive PHY so that the FEC decoder is executed on the ARM processor. The processing path is:



At B5, coded Reed–Solomon symbols leave the FPGA processing path and are provided to the PS-side decoder. After decoding on the ARM processor, the decoded message is returned at B6 and subsequent packet processing continues in the FPGA.

The substitution span is configured by the example using:

```
integrive_set_ps_span(dev, INTEGRIVE_CHAIN_RX, 5, 6);
```

The program then configures the PS buffer and timeout and registers the FEC decoder callback.

Because Reed–Solomon decoding operates on complete codewords or groups of coded symbols, the FEC example uses batch processing. User-developed PHY functions may use different processing granularities depending on the characteristics of the PHY function, as described in Chapter 11.

8.7 Verify PHY Operation

The purpose of the first demo is to confirm successful operation of the complete development and PHY processing path.

For any example, verify that:

- The application starts without an API or configuration error.
- The Integrive-100 device is successfully opened.
- The required RF and PHY configurations are accepted.
- The PHY starts successfully.
- Data transmission and/or reception proceeds as expected.
- The user-developed function, if used, is executed correctly.
- The application completes or stops normally.

For the FEC-on-PS example, verification goes one step further. The program reports:

- Number of batches processed
- Number of RS codewords processed

- Number of corrected symbols
- Number of unrecoverable codewords
- Number of bytes returned to the PL
- Byte-exact comparison with the transmitted golden payload

When a golden payload is available, the decoded message is compared codeword by codeword with the original transmitted data. This verifies that the user-developed FEC decoder not only executes on the ARM processor but also produces the expected decoded data.

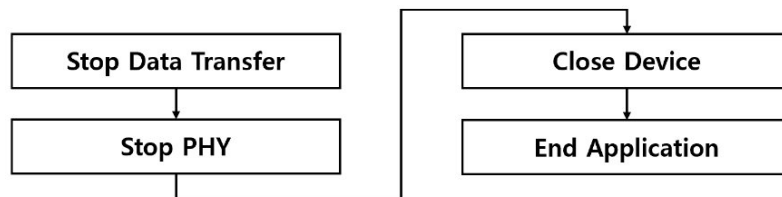
The application can also obtain PHY status using `integrive_get_status()` and report information such as received packets, CRC errors, PER, and SNR.

Note: When the FPGA FEC decoder is replaced by a PS-side implementation, FPGA-derived FEC statistics should not be used as the primary measure of the PS decoder performance. In the FEC-on-PS example, the PS-side decoding results and byte-exact comparison against the golden payload provide the primary verification.

8.8 Stop the Demo

After completing the experiment, stop the PHY and close the Integrive-100 device according to the example program.

The normal shutdown sequence is:



Parameters classified as **Stopped** should not be modified while the PHY is operating. If a structural PHY parameter needs to be changed, stop the PHY, apply the new configuration, and restart the PHY as described in Chapter 7.

8.9 Next Steps

After successfully completing the first PHY demo, users can begin modifying the example or developing their own applications and PHY functions.

Typical next steps include:

- Changing RF operating parameters
- Selecting a different supported PHY configuration
- Modifying packet parameters
- Developing application or protocol functions
- Modifying the Reed–Solomon decoder
- Evaluating another FEC algorithm on the PS
- Developing other user-defined PHY functions
- Comparing PS-side algorithms with FPGA PHY implementations

The RF and PHY configuration interfaces are described in Chapter 6, while the normal PHY operating sequence is described in Chapter 7.

Development of user-defined PHY functions and their integration into the real-time PHY processing path are described in Chapter 11.

9. Integrive SDK API

The Integrive SDK provides a C API for device access, RF and PHY configuration, real-time FPGA PHY operation, and data transmission and reception.

Applications include the public SDK header as follows:

```
#include <integrive.h>
```

This chapter summarizes the main APIs required for Integrive-100 application development. The declarations in the `integrive.h` file included with each SDK release are the authoritative definitions of function arguments, data types, structures, enumerations, and return-status values.

Detailed configuration parameters and supported ranges are described in Chapter 6.

9.1 Device Control

The device-control API is used to open and close an Integrive-100 device.

Function	Description
<code>integrive_open()</code>	Opens an Integrive-100 device and returns a device handle.
<code>integrive_close()</code>	Closes the device and releases the associated resources.

A device handle returned by `integrive_open()` is passed to subsequent RF, PHY, and runtime-control functions.

Typical usage is:

```
integrive_device_t *dev;
integrive_open(&dev);
/* Configure and operate the device */
integrive_close(dev);
```

Applications should check the return status of API calls as described in Section 9.6.

9.2 RF Configuration

The RF configuration API controls the RF transceiver parameters used by the TX and RX paths.

Function	Description
<code>integrive_rf_set_frequency()</code>	Sets the RF center frequency.
<code>integrive_rf_set_sample_rate()</code>	Sets the RF sampling rate.
<code>integrive_rf_set_bandwidth()</code>	Sets the RF bandwidth.
<code>integrive_rf_set_gain()</code>	Sets the gain for the selected TX or RX channel.
<code>integrive_rf_set_gain_mode()</code>	Sets the gain-control mode for the selected RF path.

<code>integrive_rf_enable_channel()</code>	Enables or disables the selected TX or RX channel.
--	--

The direction and channel are specified through the corresponding API arguments where applicable.

The actual function arguments and data types are defined in the released `integrive.h`. Supported RF parameter ranges and default values are described in Chapter 6.

9.3 PHY Configuration

The FPGA PHY is configured using the `integrive_phy_config_t` structure and:

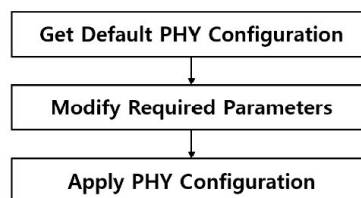
```
integrive_phy_configure(dev, &cfg);
```

The SDK also provides:

```
integrive_phy_get_default_config()
```

for obtaining the default PHY configuration as a starting point.

A typical configuration procedure is:



The PHY configuration structure contains parameters associated with modulation, FEC, OFDM, and other PHY functions.

9.3.1 Modulation

The current FPGA PHY supports:

- BPSK
- QPSK
- 16-QAM
- 64-QAM
- 256-QAM

The selected modulation mode is specified through the corresponding field of `integrive_phy_config_t`.

A fixed Gray-coded constellation mapping is used for the supported modulation schemes.

9.3.2 Forward Error Correction

The coding modes defined by the current API include:

```
INTEGRIVE_CODE_NONE
INTEGRIVE_CODE_RS_255_215
```

The FEC mode is selected through the PHY configuration interface, and Reed–Solomon processing is performed by the FPGA PHY.

Detailed FEC characteristics are described in Chapter 6.

9.3.3 OFDM Configuration

OFDM-related parameters are provided through `integrive_phy_config_t` rather than through separate setter functions.

These parameters include settings such as:

- FFT size
- Cyclic-prefix length
- Data and pilot configuration

Parameters that modify the digital PHY processing structure should be configured while the PHY is stopped.

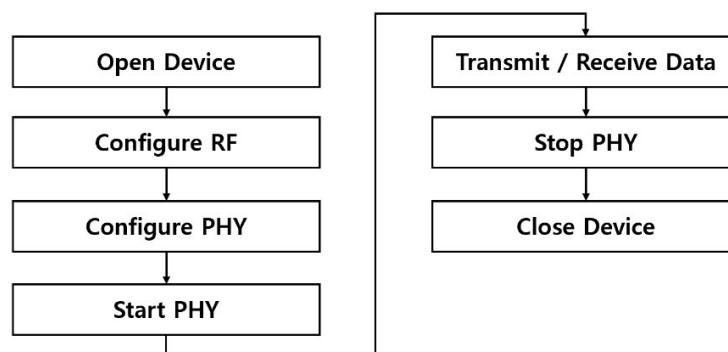
9.4 PHY Operation

After RF and PHY parameters have been configured, the real-time FPGA PHY is controlled through the following APIs.

Function	Description
<code>integrive_phy_start()</code>	Starts real-time FPGA PHY processing.
<code>integrive_phy_send()</code>	Sends application data through the PHY transmit path.
<code>integrive_phy_rcv()</code>	Receives recovered data from the PHY receive path.
<code>integrive_phy_stop()</code>	Stops FPGA PHY processing.

These functions provide the basic runtime interface for PHY experiments and application development.

A typical operating sequence is:



Once the PHY has been started, the FPGA executes the configured real-time PHY processing chain. The software application controls configuration and data transfer through the SDK interface.

9.5 User-Developed PHY Functions

Integrive-100 allows a selected FPGA PHY processing function to be replaced by a user-developed function running on the Processing System (PS).

The current SDK provides the following main interfaces for this purpose:

Function	Description
<code>integrive_phy_set_stage_impl()</code>	Selects the implementation used for a PHY processing stage.
<code>integrive_phy_get_stage_impl()</code>	Returns the currently selected implementation for a PHY processing stage.
<code>integrive_phy_set_stage_callback()</code>	Associates a user-developed software function with a selected PHY processing stage.

The SDK defines:

```
INTEGRIVE_PHY_IMPL_USER_SW
```

for selecting a user-developed software implementation where supported.

Replaceable processing stages include:

- Encoder
- Modulator
- Synchronization
- Equalizer
- Demodulator
- Decoder

In the current implementation, **one FPGA PHY processing block at a time** can be replaced by a user-developed function running on the PS.

The remaining PHY functions continue to operate through the standard FPGA PHY processing chain.

The detailed development and streaming-processing model for user-developed PHY functions is described in Chapter 11.

9.6 API Return Status

Integrive SDK functions use `integrive_status_t` to report the result of an API operation.

`INTEGRIVE_OK` indicates successful completion. A negative return value indicates an error condition defined by the installed SDK release.

Applications should check the return status of SDK calls, particularly during device initialization and RF/PHY configuration.

For example:

```
integrive_status_t status;
status = integrive_phy_start(dev);
if (status != INTEGRIVE_OK) {
    /* Handle the error */
}
```

The SDK also provides:

```
integrive_strerror(status);
```

to obtain a human-readable description of an error status.

Detailed error handling is described in Chapter 10.

9.7 API Version and Compatibility

The public API definitions in `integrive.h` correspond to the installed Integrive SDK release.

Applications should be built using the header and library files supplied with the same SDK release.

When the SDK is updated, users should refer to the accompanying release notes for changes to:

- Supported API functions
- Configuration parameters
- Data structures and enumerations
- FPGA PHY compatibility

The `integrive.h` file supplied with the installed SDK should be treated as the authoritative reference for function declarations, structures, enumerations, and return-status definitions.

10. Error Handling

Integrive SDK functions return an `integrive_status_t` value to indicate whether an operation completed successfully.

`INTEGRIVE_OK` indicates successful completion. A negative return value indicates an error condition.

Applications should check the return status of SDK calls, particularly during device initialization, RF and PHY configuration, and PHY startup.

The status-code definitions in the `integrive.h` file supplied with the installed SDK are the authoritative definitions for the corresponding SDK release.

10.1 Checking API Results

The return status of an SDK operation should be checked before the application proceeds to the next operation.

A typical error-checking pattern is:

```
integrive_status_t status;
```

```
status = integrive_phy_start(dev);
if (status != INTEGRIVE_OK) {
    /* Handle the error */
}
```

The SDK provides:

```
integrive_strerror(status);
```

to obtain a human-readable description of an error status.

For example:

```
if (status != INTEGRIVE_OK) {
    printf("Integrive error: %s\n",
        integrive_strerror(status));
}
```

This allows applications to report an error without duplicating the SDK status-code definitions.

10.2 Configuration Errors

RF and PHY configuration requests are validated before they are applied.

If a requested parameter or configuration is invalid or unsupported, the API returns an error status. The application should correct the configuration before continuing.

For configuration errors:

- Do not assume that an invalid request has been automatically adjusted.
- Check the returned `integrive_status_t`.
- Use `integrive_strerror()` to identify the reported error.
- Correct the configuration before starting or restarting the PHY.

Detailed supported parameter ranges and configuration rules are described in Chapter 6.

10.3 Recommended Error-Handling Flow

For basic Integrive-100 applications, the recommended error-handling procedure is shown in Figure 10-1. Each SDK function call should be followed by a return-status check before the application proceeds to the next operation.

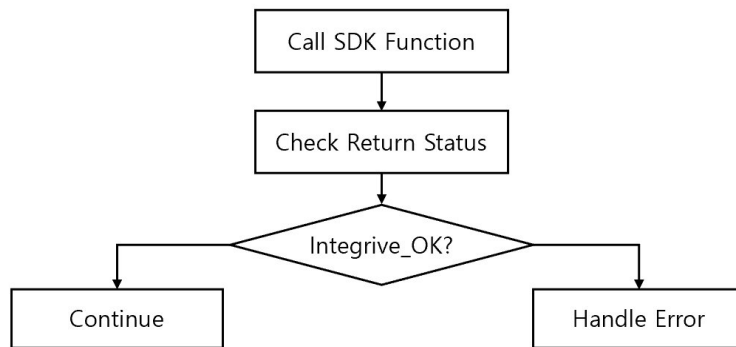
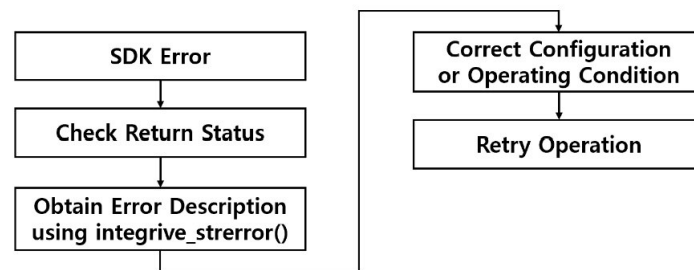


Figure 10-1. Recommended SDK Error-Handling Flow

When an error occurs during device initialization, RF/PHY configuration, or PHY startup, the application should resolve the error before continuing the experiment.

The general procedure is:



For repeated or unexpected errors, users should verify that the application is built against the header and library files supplied with the installed SDK release.

10.4 Status-Code Reference

Individual status-code definitions are not duplicated in this User Manual because they may evolve as the SDK is updated.

The installed:

`integrive.h`

should be used as the authoritative reference for:

- Status-code names
- Status-code values
- Function declarations
- API data types

This ensures that application software uses error definitions corresponding to the installed SDK version.

11. PHY Research Workflow

Integrive-100 provides a research environment in which user-developed PHY algorithms can be evaluated together with the built-in real-time FPGA PHY using actual RF signals.

Unlike a host-based SDR approach in which received data is transferred to a host PC for

subsequent processing, Integrive-100 allows a selected user-developed PHY function to participate directly in the PHY processing chain.

This enables researchers to develop and verify an algorithm in MATLAB, Python, or C/C++, integrate the resulting PHY function with the existing Integrive FPGA PHY, and evaluate it through real-time RF experiments.

11.1 PHY Algorithm Research Workflow

A typical PHY research workflow using Integrive-100 is shown in Figure 11-1.

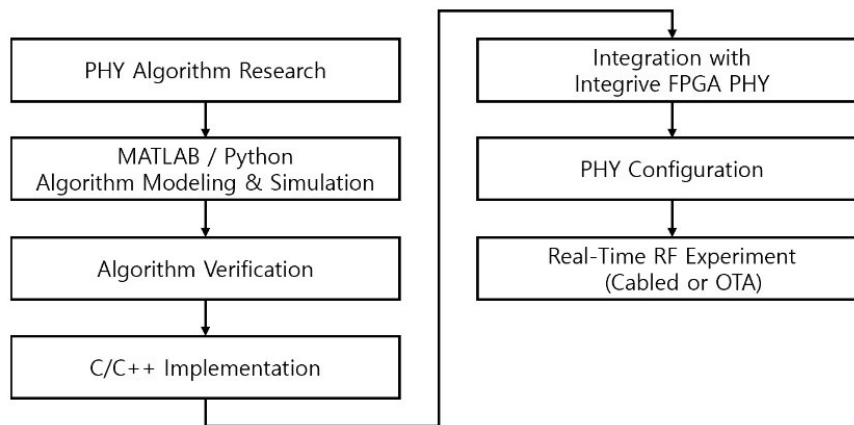


Figure 11-1. Integrive-100 PHY Algorithm Research Workflow

MATLAB / Python Algorithm Modeling and Simulation

A researcher may first develop a new PHY algorithm using MATLAB or Python. At this stage, the algorithm can be modeled and evaluated through simulation before being tested with actual RF signals.

- **Algorithm Verification**

The researcher verifies the functionality and performance of the proposed algorithm using the simulation model.

This step allows the algorithm to be evaluated and refined before implementation on the Integrive-100 platform.

- **C/C++ Implementation**

After algorithm verification, the selected PHY function can be implemented in C/C++ for execution on the Integrive-100 Processing System (PS).

Where appropriate, numerical representation and computational complexity can be adapted to meet the requirements of the target experiment.

- **Integration with the Integrive FPGA PHY**

The user-developed PHY function is integrated with the existing Integrive FPGA PHY processing chain.

The researcher does not need to rebuild the complete FPGA PHY. The remaining built-in PHY functions continue to operate in the FPGA while the selected user-developed function executes on the PS.

For example, a user-developed FEC function can operate together with the built-in

synchronization, OFDM, channel estimation/equalization, and modulation/demodulation functions.

- **PHY Configuration**

The required RF and PHY parameters are configured through the Integrive SDK. The selected processing stage is configured to use the user-developed software function while the remaining stages continue to use the standard FPGA implementation.

The SDK interfaces for user-developed PHY functions are described in **Section 9.5**.

- **Real-Time RF Experiment**

After configuration, the researcher can evaluate the algorithm using actual RF signals.

Experiments may be performed using a cabled RF connection or over-the-air (OTA) transmission. This enables an algorithm initially evaluated through simulation to be tested under actual communication-channel conditions.

11.2 Streaming PHY Processing

A key characteristic of the Integrive-100 research model is **where and when a user-developed PHY function participates in the PHY processing chain**.

A conventional host-based SDR typically uses a frame-oriented processing model. Received data is first collected as a complete frame and then transferred to a host PC, where subsequent processing is performed in host software.

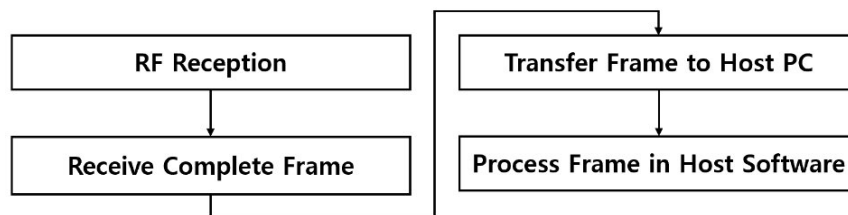


Figure 11-2. Frame-Oriented Processing in a Conventional Host-Based SDR

In contrast, Integrive-100 allows a user-developed PHY function running on the Processing System (PS) to participate directly in the PHY processing chain. Data produced by a preceding FPGA PHY function can be transferred to the PS for user-defined processing, and the processed data can then be returned to the FPGA so that the remaining PHY functions continue processing.

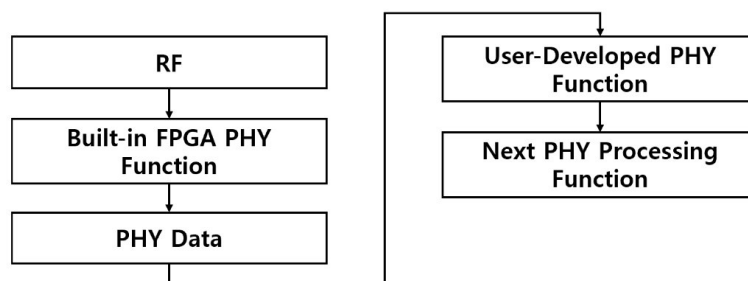


Figure 11-3. Streaming PHY Processing with a User-Developed PHY Function

This approach does not require a complete received frame to be transferred to an external host PC before the user-developed PHY function can operate. Instead, the user-developed function

participates at the appropriate point in the PHY processing chain while the remaining real-time PHY functions continue to operate in the FPGA.

Processing Granularity

The processing granularity of a user-developed PHY function depends on the characteristics of the PHY algorithm being implemented.

Different PHY functions naturally operate on different processing units. For example, OFDM processing may operate on OFDM symbols, while an FEC decoder operates on FEC codewords. Other PHY algorithms may require different amounts or structures of input data before processing can begin.

Therefore, the processing unit of a user-developed PHY function is not required to correspond directly to an OFDM symbol. The relationship between OFDM symbols and FEC codewords, for example, depends on the PHY and FEC configuration and is not necessarily one-to-one.

A user-developed PHY function may accumulate the amount of input data required by its algorithm, perform the processing operation, and return the resulting data to the PHY processing chain.

Example: User-Developed FEC

A user-developed FEC decoder illustrates this processing model.

The FEC decoder receives coded data from the preceding FPGA PHY processing stage. The decoder operates when sufficient input data has been received for the processing unit required by the FEC algorithm. After decoding, the resulting data is returned to the FPGA PHY processing chain, where subsequent processing continues.

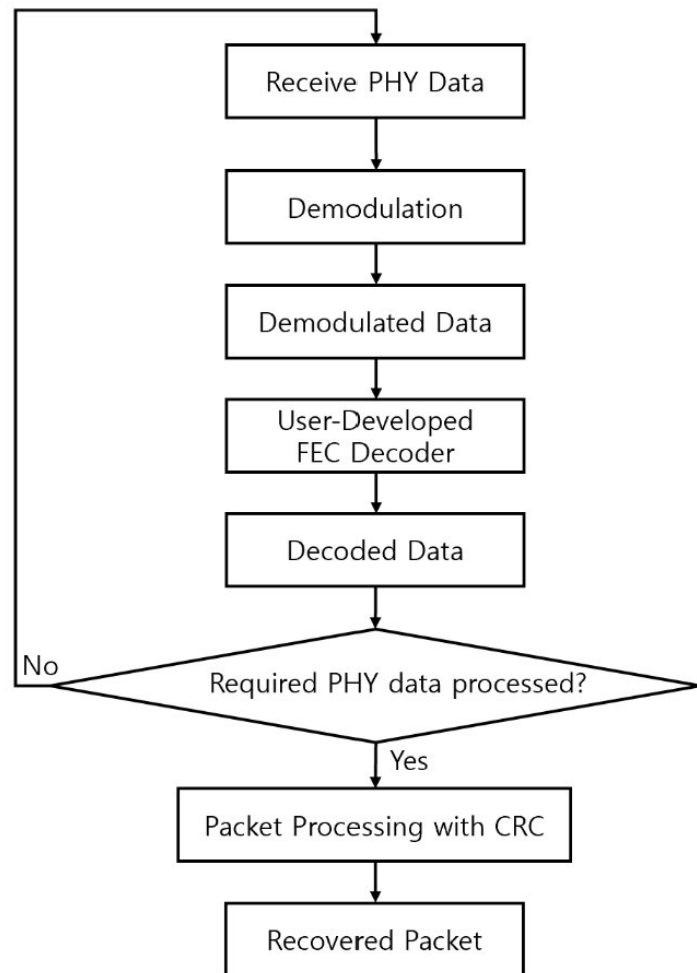


Figure 11-4. Streaming PHY Processing with a User-Developed FEC Decoder

The FEC decoder therefore operates according to its own processing requirements while participating directly in the Integrive-100 streaming PHY processing path. The relationship between OFDM symbols and FEC codewords depends on the PHY and FEC configuration and is not fixed by the Integrive-100 processing model.

Figure 11-5 illustrates this input-dependent processing behavior. The FEC decoder may accumulate input data until the amount required by the FEC algorithm is available. Once the required input is available, the decoder processes the data and returns the decoded result to the FPGA PHY processing chain.

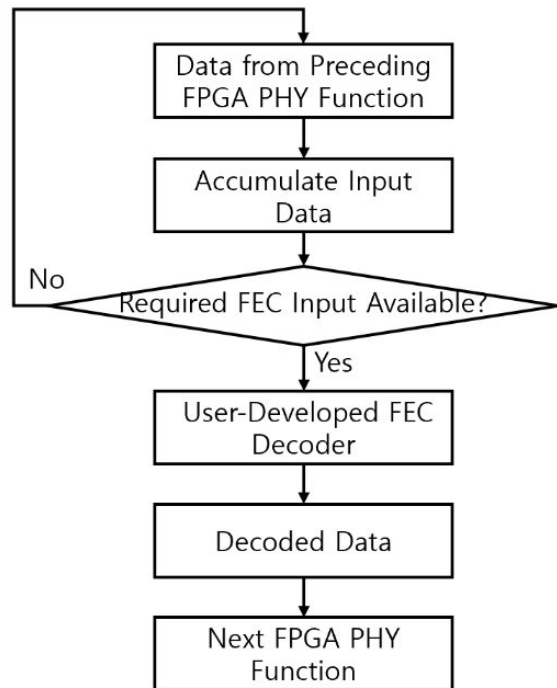


Figure 11-5. Input-Dependent Processing with a User-Developed FEC Decoder

This model allows user-developed PHY algorithms to be evaluated together with the remaining FPGA PHY functions using real RF signals, without requiring the complete PHY to be reimplemented in software or the complete received frame to be processed on an external host PC.

11.3 Development Scope

Integrive-100 supports PHY research at both the **configuration level** and the **user-developed function level**.

Researchers can configure supported RF and PHY parameters through the Integrive SDK and can develop selected PHY functions for integration with the existing real-time PHY processing chain.

The standard Integrive-100 SDK does not provide FPGA PHY source code or support user modification or rebuilding of individual FPGA PL blocks.

Development Type	Integrive-100 v1
RF/PHY parameter configuration	Supported
User application/protocol development	Supported
User-developed PHY function on PS	Supported
Real-time cabled/OTA experiments	Supported
FPGA PHY RTL modification	Not supported
Individual FPGA PL block replacement by user RTL	Not supported

The supported user-developed PHY stages and their SDK interfaces are described in Section 9.5.

12. Specifications and Limitations

This chapter summarizes the operating specifications and limitations of the current Integrive-100 release. Only parameters supported by the current hardware, FPGA PHY, and SDK should be used when configuring the platform.

Detailed configuration information is provided in Chapter 6.

12.1 Operating Specifications

The principal operating specifications of Integrive-100 are summarized below.

Parameter	Supported Range / Value
RF Frequency	300 MHz – 6 GHz
Sample Rate	Up to 61.44 MSPS
RF Bandwidth	Up to 20 MHz
RF Channels	Up to 2 TX / 2 RX
Modulation	BPSK, QPSK, 16-QAM, 64-QAM, 256-QAM
FEC	Reed–Solomon
Payload Length	100 – 6450 bytes

The current default configuration is:

Parameter	Default Value
RF Frequency	915 MHz
Sample Rate	60 MSPS
RF Bandwidth	20 MHz
FFT Size	1024
CP Length	26 samples
TX / RX Configuration	1 TX / 1 RX
Packet Payload Length	1500 bytes
RX Gain Mode	AGC

The data and pilot allocation is user-configurable. The configuration of **264 data subcarriers and 24 pilot subcarriers** described in this manual is a **reference configuration**, not a system default or fixed hardware property.

12.2 Current Limitations

The standard Integrive-100 SDK provides access to the supported RF and PHY configuration and

operation functions but does not provide the FPGA PHY source code.

Users cannot modify or rebuild individual FPGA PL blocks through the standard SDK. Instead, a selected FPGA PHY processing function can be replaced by a user-developed C/C++ function running on the Processing System (PS), while the remaining PHY functions continue to operate in the FPGA.

In the current implementation, **one FPGA PHY processing block at a time** can be replaced by a user-developed PS function.

The supported user-developed PHY stages and their SDK interfaces are described in Section 9.5, and the PHY research workflow is described in Chapter 11.

Parameters that modify the digital PHY processing structure must be changed while the PHY is stopped. Selected RF parameters may be changed during operation where supported.

The configuration rules are described in Chapter 6, and the operating and reconfiguration procedures are described in Chapter 7.

13. Troubleshooting

This chapter provides basic troubleshooting guidance for common problems encountered when operating Integrive-100. For SDK return-status handling and error reporting, refer to Chapter 10.

13.1 Common Problems and Recommended Actions

Symptom	Possible Cause	Recommended Action
Cannot connect to Integrive-100 from the host PC	The board may not be powered correctly, or the network connection may not be available.	Check the board power and network connection. Confirm that the host PC can communicate with Integrive-100.
User application or kernel module does not run correctly	The application may not have been built for the target environment, or the required Integrive driver may not be loaded.	Verify the host build configuration and confirm that integrive100.ko is loaded on Integrive-100.
integrive_open() fails	The Integrive device or required software component may not be available to the application.	Check the SDK installation and verify that the required Integrive driver is loaded. Check the returned status and use integrive_strerror() for additional information.
PHY configuration does not take effect	The requested parameter may require the PHY to be stopped before reconfiguration.	Stop the PHY using integrive_phy_stop(), apply the required configuration, and restart the PHY. Refer to Chapters 6 and 7 for parameter-change rules.
PHY does not start	One or more RF or PHY parameters may be invalid or unsupported.	Check the return status from integrive_phy_start() and verify the RF and PHY configuration.

No data is received	TX and RX RF or PHY configurations may not match.	Verify the center frequency, sample rate, RF bandwidth, modulation, FEC, FFT size, CP length, and other required PHY parameters at both ends.
Cabled TX/RX test does not operate correctly	The received signal level may be excessive, or the TX/RX configurations may not match.	Use appropriate RF attenuation between TX and RX and verify compatible RF and PHY configurations before transmission.
Received data contains frequent errors	RF conditions may be poor, or TX/RX PHY configurations may not be compatible.	Verify that both ends use compatible RF and PHY parameters. For OTA tests, check the antenna setup, RF environment, and received signal conditions.
User-developed PHY function does not operate as expected	The selected PHY stage, implementation mode, or user callback may not be configured correctly.	Verify the user-developed PHY stage configuration and interface described in Section 9.5. Begin with a built-in FPGA PHY configuration before enabling the user-developed function.

13.2 Basic Troubleshooting Procedure

When an experiment does not operate as expected, check the system in the following order:

1. Confirm that Integrive-100 is powered and accessible from the host PC.
2. Confirm that the required Integrive driver is loaded.
3. Confirm that the SDK and user application are installed or transferred correctly.
4. Confirm that `integrive_open()` completes successfully.
5. Verify the RF configuration.
6. Verify the PHY configuration.
7. Start the PHY and perform a basic TX/RX test.
8. If a user-developed PHY function is being used, first verify operation with the corresponding built-in FPGA PHY function.
9. If the problem persists, record the SDK version, configuration, application output, and returned status before requesting technical support.

For initial verification, begin with the **default configuration described in Chapter 6** before modifying RF or PHY parameters.

14. Regulatory and Safety Notice

Integrive-100 is a software-defined radio (SDR) development platform intended for research, education, and wireless communication development.

Users are responsible for operating the platform in accordance with applicable regulations and for observing appropriate RF and electrical safety practices.

14.1 RF Operation

Radio-frequency transmission is subject to the spectrum regulations applicable in the country or region where Integrive-100 is operated.

The frequencies used in examples in this manual, including 915 MHz, are provided for demonstration and reference purposes only. They do not imply that transmission at those frequencies is permitted in every country or region.

Users are responsible for selecting operating frequencies, transmit power levels, bandwidths, antennas, and other RF conditions permitted by applicable local regulations.

For initial testing and development, a cabled RF connection with appropriate attenuation or a suitably controlled RF test environment is recommended.

14.2 Cabled TX/RX Testing

When performing cabled TX/RX testing, use appropriate RF attenuation between the TX and RX ports to prevent excessive signal levels at the receiver input.

Before enabling transmission, verify that the expected received signal level is within the allowable operating range of the receiver.

When connecting Integrive-100 to external RF test equipment, ensure that the signal level applied to each device is within the allowable input range specified for that equipment.

Avoid applying excessive RF power to the Integrive-100 RX port or to connected RF test equipment.

14.3 Handling Precautions

Observe standard electrostatic discharge (ESD) precautions when handling Integrive-100.

Stop RF transmission before connecting or disconnecting RF cables whenever practical.

Use only the specified power supply and recommended connection methods for Integrive-100.

Do not operate the platform if the board, enclosure, power connection, or RF connectors show visible signs of damage.

Glossary

Term	Meaning
AGC	Automatic gain control; automatic adjustment of receiver gain according to the received signal level
API	Application programming interface used by software to configure and control Integrive-100
ARM	Processor architecture used by the Processing System (PS) of Integrive-100
BER	Bit error rate; the ratio of incorrectly received bits to the total number of transmitted or received bits evaluated
BPSK	Binary phase-shift keying
CP	Cyclic prefix; samples appended to an OFDM symbol to reduce the effects of multipath delay spread
CRC	Cyclic redundancy check used for packet error detection
EVM	Error vector magnitude; a measure of modulation signal quality
FEC	Forward error correction
FFT / IFFT	Fast Fourier transform and inverse fast Fourier transform used in OFDM processing
FPGA	Field-programmable gate array used for real-time PHY processing
LO	Local oscillator used to set the RF center frequency
OFDM	Orthogonal frequency-division multiplexing
OTA	Over-the-air; wireless transmission or experimentation using radiated RF signals
PER	Packet error rate; the ratio of incorrectly received packets to the total number of packets evaluated
PHY	Physical layer; the communication processing layer responsible for signal transmission and reception
PL	Programmable Logic; the FPGA portion of the Zynq device used for real-time PHY processing
PS	Processing System; the ARM processor portion of the Zynq device used for software execution and control
QAM	Quadrature amplitude modulation
QPSK	Quadrature phase-shift keying
RF	Radio frequency
RS(n,k)	Reed–Solomon code with n code symbols per codeword and k data symbols
SDK	Software development kit containing the software interfaces and development resources for Integrive-100
SDR	Software-defined radio
SNR	Signal-to-noise ratio
SPS	Samples per second